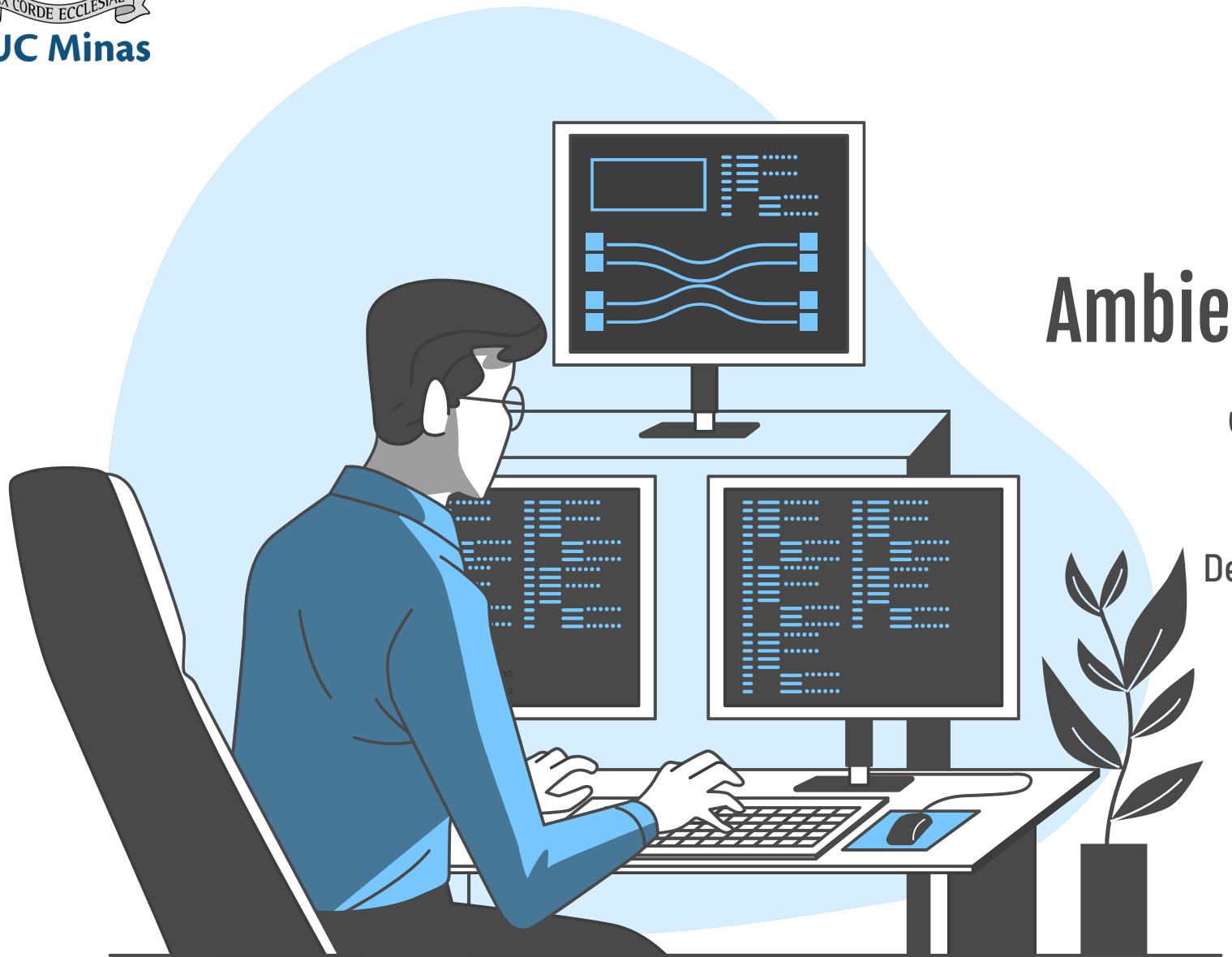
PUC Minas



Aula 01 · Back-end

Ambiente de desenvolvimento: JDK 25, Maven e VS Code

Desenvolvimento e Integração de Aplicações Web

Prof. Dr. João Paulo Aramuni

Engenharia de Software · PUC Minas

github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web

Apresentação da aula

Nesta aula você vai

1. Diferenciar **JDK**, **JRE** e **JVM**
2. Instalar o **JDK 25** e configurar o `JAVA_HOME`
3. Entender o papel do **Maven** e do **Maven Wrapper** (`mvnw`)
4. Preparar o **VS Code** com as extensões de Java e Spring Boot
5. Rodar um primeiro programa Java pelo terminal

Antes de começar

- Computador com Windows, macOS ou Linux
- Acesso de administrador para instalar programas
- Conta no GitHub (usaremos nas próximas pastas)

Duração sugerida

100 min

Código da disciplina

Projetos de referência na pasta PROJETOS do GitHub

Roteiro

01 O que é o back-end

02 JDK, JRE e JVM

03 Por que o JDK 25

04 Instalando o JDK 25

05 Maven: o gerenciador do projeto Java

06 Preparando o VS Code

07 Primeiro programa: testando o JDK

08 Checklist do ambiente

09 Aprofundando: a JVM e o ciclo de build

10 Você já conhece o npm: agora o Maven

01

O que é o back-end

O que é o back-end

Toda aplicação web tem duas metades. O **front-end** é o que roda no navegador do usuário: telas, botões, formulários. O **back-end** é o programa que roda em um servidor, recebe pedidos (requisições HTTP), aplica as regras de negócio, conversa com o banco de dados e devolve uma resposta, quase sempre em **JSON**.

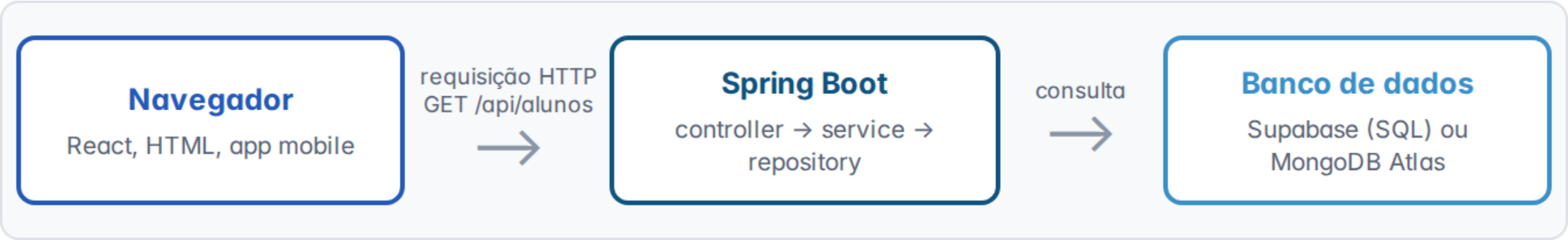
Nesta pasta vamos construir back-ends com **Java** e **Spring Boot**. O Spring Boot é um framework que já traz um servidor web embutido (o Tomcat), integração com bancos de dados, segurança e muito mais, tudo configurado com o mínimo de código.

01

O que é o back-end

Aula 01 · Back-end

O que é o back-end



O back-end fica no meio: recebe a requisição, processa e conversa com o banco.

02

JDK, JRE e JVM

JDK, JRE e JVM

Essas três siglas aparecem o tempo todo e costumam confundir. Pense nelas como caixas, uma dentro da outra:

Sigla	Nome	Para que serve
JVM	Java Virtual Machine	Executa o *bytecode* (`.class`). É ela que faz o Java rodar em qualquer sistema operacional.
JRE	Java Runtime Environment	JVM + bibliotecas padrão. Suficiente para **rodar** um programa Java pronto (um `.jar`).
JDK	Java Development Kit	JRE + ferramentas de desenvolvimento (`javac`, `jar`, `jshell`…). Necessário para **compilar**.

JDK, JRE e JVM

O fluxo é sempre o mesmo: você escreve `Aluno.java`, o compilador `javac` (que vem no JDK) gera `Aluno.class`, e a JVM executa esse arquivo. No dia a dia, o Maven faz essa compilação por você.

02

JDK, JRE e JVM

Aula 01 · Back-end

JDK, JRE e JVM

Observação

Em produção (no Render, por exemplo) só precisamos da **JRE** para rodar o `.jar`. Por isso, na pasta Deploy, o Dockerfile usa uma imagem com JDK para compilar e outra, menor, só com JRE para executar.

03

Por que o JDK 25

Por que o JDK 25

O Java lança uma versão nova a cada seis meses, e a cada dois anos uma delas é **LTS** (*Long-Term Support*), com anos de correções garantidas. O **JDK 25** (setembro de 2025) é a LTS mais recente, sucessora do 21 e do 17. O Spring Boot 4 exige no mínimo Java 17 e é totalmente compatível com o 25.

Alguns recursos modernos que você vai encontrar nos projetos:

03

Por que o JDK 25

Aula 01 · Back-end

Por que o JDK 25

Recurso	Exemplo	Desde
`record` (classe de dados imutável)	`record AlunoDTO(String nome, String email) {}`	Java 16
`var` (inferência de tipo)	`var lista = new ArrayList();`	Java 10
Text blocks	`String sql = """ SELECT * FROM aluno """;`	Java 15
Pattern matching no `switch`	`case Aluno a -> a.getNome();`	Java 21
Virtual threads	`spring.threads.virtual.enabled=true`	Java 21
Método `main` simplificado	`void main() { IO.println("Olá"); }`	Java 25

04

Instalando o JDK 25

Instalando o JDK 25

Use uma distribuição gratuita do OpenJDK. A mais comum é o **Eclipse Temurin** (site *adoptium.net*), a mesma das imagens Docker que usaremos no deploy.

1. Acesse **adoptium.net**, escolha **Temurin 25 (LTS)**, seu sistema operacional e a arquitetura (x64 ou aarch64 para Mac com chip Apple).
2. **Windows:** baixe o `.msi` e, no instalador, marque as opções **Set JAVA_HOME variable** e **Add to PATH**.
3. **macOS:** baixe o `.pkg` ou use o Homebrew: `brew install --cask temurin@25`.
4. **Linux (Ubuntu):** `sudo apt install openjdk-25-jdk` ou use o SDKMAN (abaixo).
5. Feche e abra o terminal de novo e confira a instalação com `java -version` e `javac -version`.

Instalando o JDK 25

Terminal

```
aluno@pc:~/projeto$ java -version
openjdk version "25.0.1" 2025-10-21 LTS
OpenJDK Runtime Environment Temurin-25.0.1+8 (build 25.0.1+8-LTS)
OpenJDK 64-Bit Server VM Temurin-25.0.1+8 (build 25.0.1+8-LTS, mixed mode)
aluno@pc:~/projeto$ javac -version
javac 25.0.1
aluno@pc:~/projeto$ echo $JAVA_HOME
/usr/lib/jvm/temurin-25-jdk-amd64
```

Conferindo a instalação no terminal. O número exato da versão de correção pode ser maior no seu computador.

Instalando o JDK 25

Dica

Precisa ter mais de uma versão do Java (17 para um projeto antigo, 25 para os novos)? Use o **SDKMAN** no Linux/macOS:

`sdk install java 25-tem` e depois `sdk use java 25-tem`. No Windows, ajuste o `JAVA_HOME` nas *Variáveis de Ambiente* do sistema.

O que é o JAVA_HOME

`JAVA_HOME` é uma **variável de ambiente** que aponta para a pasta onde o JDK foi instalado. O Maven, o VS Code e o Spring Boot leem essa variável para saber qual Java usar. Se ela estiver errada, você verá erros como *"JAVA_HOME is not defined correctly"* ao rodar `./mvnw`.

04

Instalando o JDK 25

Aula 01 · Back-end

O que é o JAVA_HOME

Configurando o JAVA_HOME

```
# Windows (PowerShell) - mostra o valor
echo $env:JAVA_HOME
# Windows - define para o usuário (abra outro terminal depois)
setx JAVA_HOME "C:\Program Files\Eclipse Adoptium\jdk-25.0.1.8-hotspot"

# macOS (zsh) - adicione ao ~/.zshrc
export JAVA_HOME=$(/usr/libexec/java_home -v 25)

# Linux (bash) - adicione ao ~/.bashrc
export JAVA_HOME=/usr/lib/jvm/temurin-25-jdk-amd64
export PATH=$JAVA_HOME/bin:$PATH
```

05

Maven: o gerenciador do projeto Java

Maven: o gerenciador do projeto Java

Um projeto Spring Boot depende de dezenas de bibliotecas (*dependências*). Baixar cada `.jar` à mão seria inviável. O **Maven** resolve isso: você declara o que precisa no arquivo `pom.xml` e ele baixa tudo do repositório central (*Maven Central*), compila, roda os testes e empacota o projeto em um `.jar`.

05

Maven: o gerenciador do projeto Java

Aula 01 · Back-end

Maven: o gerenciador do projeto Java

Comando	O que faz
`mvn compile`	Compila o código de `src/main/java`
`mvn test`	Compila e roda os testes de `src/test/java`
`mvn package`	Gera o `.jar` dentro da pasta `target/`
`mvn clean`	Apaga a pasta `target/` (começa do zero)
`mvn spring-boot:run`	Sobe a aplicação Spring Boot

Maven Wrapper (`mvnw`)

Você **não precisa instalar o Maven**. Todo projeto gerado pelo Spring Initializr vem com o *Maven Wrapper*: os scripts `mvnw` (Linux/macOS) e `mvnw.cmd` (Windows) e a pasta `.mvn/wrapper`. Na primeira execução ele baixa a versão certa do Maven sozinho. Assim, todo mundo da turma usa exatamente a mesma versão.

05

Maven: o gerenciador do projeto Java

Aula 01 · Back-end

Maven Wrapper (`mvnw`)

Terminal

```
aluno@pc:~/projeto/Back$ ./mvnw spring-boot:run
[INFO] Scanning for projects...
[INFO] --- spring-boot:4.1.1:run (default-cli) @ Back ---
  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
Tomcat started on port 8080 (http) with context path '/'
Started BackApplication in 2.913 seconds (process running for 3.2)
```

Subindo um projeto com o Maven Wrapper. No Windows use `mvnw.cmd spring-boot:run` ou `.\mvnw spring-boot:run` no PowerShell.

Maven Wrapper (`mvnw`)

Erro comum

`permission denied: ./mvnw` no Linux/macOS: o arquivo perdeu a permissão de execução (comum ao baixar o projeto em .zip). Resolva com `chmod +x mvnw`.

06

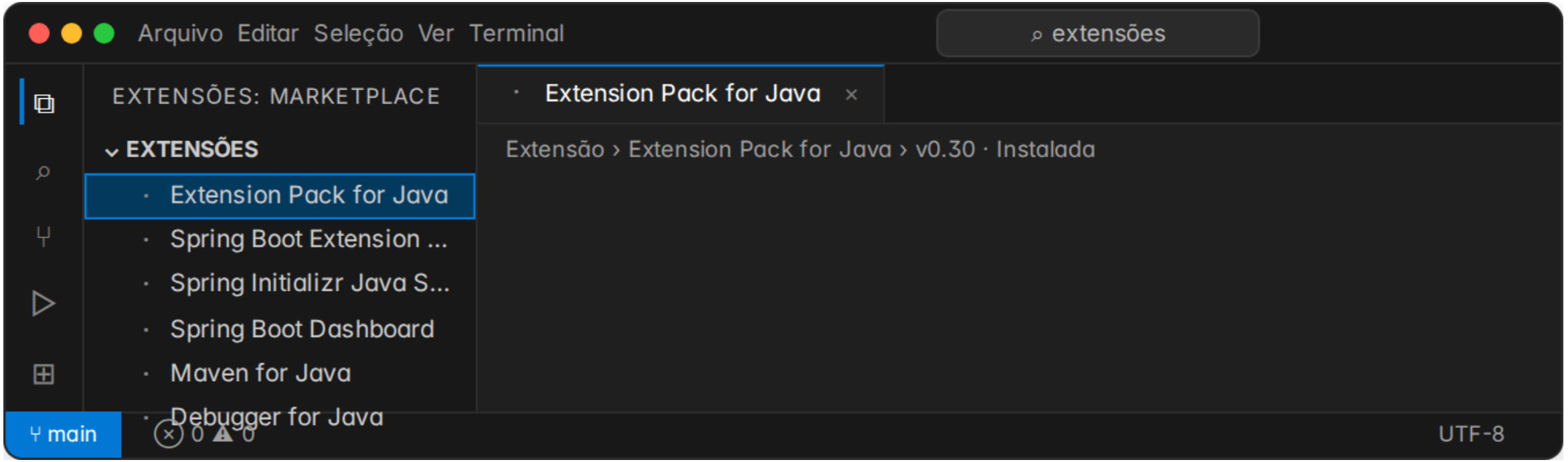
Preparando o VS Code

Preparando o VS Code

O VS Code é um editor leve que vira uma IDE completa de Java com extensões. Instale pela aba **Extensões** (`Ctrl+Shift+X`):

Extensão	Para que serve
Extension Pack for Java (Microsoft)	Linguagem Java, depuração, testes, Maven e o painel *Java Projects*
Spring Boot Extension Pack (VMware/Broadcom)	Spring Initializr, *Spring Boot Dashboard*, autocompletar no `application.properties`
ES7+ React snippets e **ESLint**	Para a pasta Front-end
Docker ou **Container Tools** (Microsoft)	Para a pasta Deploy: realce e validação do Dockerfile

Preparando o VS Code



Aba **Extensões** com os pacotes de Java e Spring Boot instalados. Pesquise pelo nome e clique em **Install**.

Preparando o VS Code

Dica

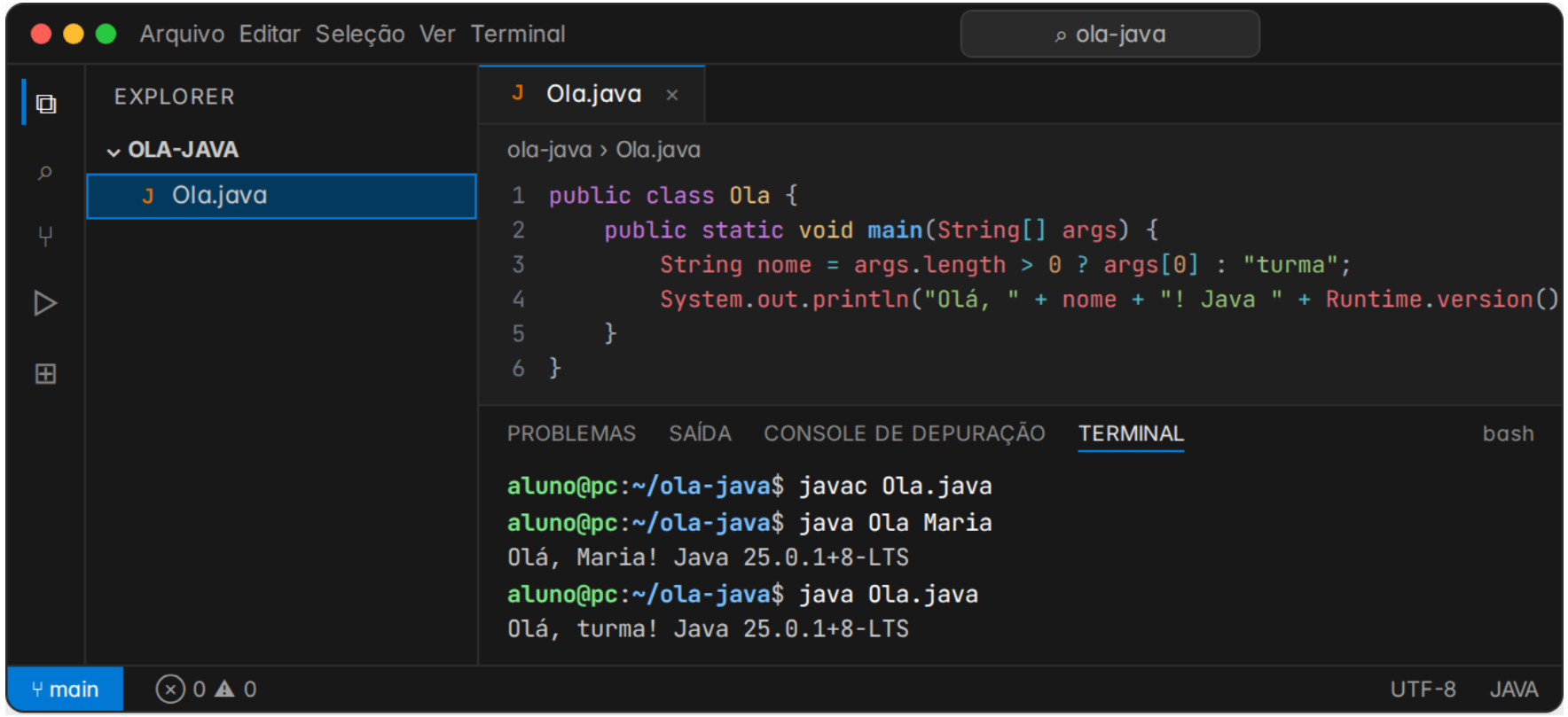
Depois de instalar, abra a paleta de comandos com `Ctrl+Shift+P` e rode **Java: Configure Java Runtime**. A tela mostra qual JDK o VS Code encontrou. Ele deve listar o **25**.

07

Primeiro programa: testando o JDK

Primeiro programa: testando o JDK

Antes do Spring, vamos garantir que o Java compila e executa. Crie uma pasta, abra no VS Code e crie o arquivo `Ola.java`:



Primeiro programa Java aberto no editor, com o terminal integrado (`Ctrl+'`) mostrando a compilação e a execução.

Primeiro programa: testando o JDK

Repare na última linha: desde o Java 11 é possível rodar `java Ola.java` direto, sem `javac`. É prático para testes, mas projetos reais usam o Maven.

07

Primeiro programa: testando o JDK

Aula 01 · Back-end

08

Checklist do ambiente

Checklist do ambiente

Antes da próxima aula, os quatro comandos abaixo precisam funcionar no seu terminal:

Checklist

```
java -version     # deve mostrar 25.x
javac -version    # deve mostrar 25.x
git --version     # usaremos para enviar o código ao GitHub
code --version    # VS Code acessível pelo terminal (abre pastas com: code .)
```

09

Aprofundando: a JVM e o ciclo de build

O que acontece quando o Java executa

1. Compilação

`javac` transforma `.java` em **bytecode** (`.class`), independente de sistema operacional.

2. Carregamento

O *ClassLoader* da JVM lê as classes do `.jar` sob demanda, só quando são usadas.

3. Interpretação + JIT

A JVM interpreta o bytecode e o compilador **JIT** converte os trechos mais usados em código de máquina.

4. Garbage Collector

Objetos sem referência são removidos da memória automaticamente. No Java 25 o padrão é o **G1**.

Aprofundando: a JVM e o ciclo de build

Você pode ver o bytecode com o `javap`, que vem no JDK. Não é preciso decorar instruções; o objetivo é perceber que o `.class` é um formato intermediário, e é ele que viaja dentro do `.jar` até o Render.

Terminal

```
$ javac Ola.java
$ javap -c Ola
public class Ola {
  public static void main(java.lang.String[]);
    Code:
       0: aload_0
       1: arraylength
       2: ifle          11
       ...
      29: invokevirtual #27  // Method java/io/PrintStream.println
      32: return
}
```

Aprofundando: a JVM e o ciclo de build

Fase do Maven	O que faz	Comando
validate	Confere se o `pom.xml` está correto	`./mvnw validate`
compile	Compila `src/main/java` para `target/classes`	`./mvnw compile`
test	Compila e roda `src/test/java`	`./mvnw test`
package	Gera o `.jar` em `target/`	`./mvnw package`
verify	Roda verificações extras (integração, cobertura)	`./mvnw verify`
install	Copia o `.jar` para o repositório local `~/.m2`	`./mvnw install`

Aprofundando: a JVM e o ciclo de build

As fases são **cumulativas**: pedir `package` executa `validate`, `compile` e `test` antes. Por isso o `-DskipTests` aparece tanto no Dockerfile: ele pula a fase de testes, que tentaria conectar no banco.

09

Aprofundando: a JVM e o ciclo de build

Aula 01 · Back-end

Aprofundando: a JVM e o ciclo de build

Observação

Toda dependência baixada pelo Maven fica em `~/.m2/repository` (no Windows, `C:\Users\\.m2`). É por isso que o segundo projeto Spring Boot que você cria compila muito mais rápido: as bibliotecas já estão no seu computador. Se uma dependência corromper, apague a pasta dela ali e rode o build de novo.

Aprofundando: a JVM e o ciclo de build

Atalho no VS Code	Ação
`Ctrl+Shift+P`	Paleta de comandos (tudo começa aqui)
`Ctrl+P`	Abrir arquivo pelo nome
`` `Ctrl+` ``	Mostrar/esconder o terminal integrado
`F5`	Iniciar depuração
`F12`	Ir para a definição de uma classe ou método
`Shift+Alt+F`	Formatar o arquivo
`Ctrl+.`	Correções rápidas (importar classe, criar método)

Depurando com breakpoints

1. Clique à esquerda do número da linha para criar um **breakpoint** (bolinha vermelha).
2. Inicie com **F5** ou com o botão **Debug** que aparece acima do `main`.
3. Quando a execução parar, examine as variáveis no painel **Variables** e passe o mouse sobre elas no código.
4. Use **F10** (próxima linha), **F11** (entrar no método) e **F5** (continuar).
5. Em projetos Spring, faça uma requisição pelo navegador e veja o breakpoint do controller ser atingido.

Versões LTS x versões intermediárias

LTS (17, 21, 25)

- Correções de segurança por anos
- Padrão em empresas e imagens Docker
- Versão escolhida pela disciplina: **25**

vs

Intermediárias (22, 23, 24, 26)

- Suporte de apenas 6 meses
- Boas para experimentar recursos novos
- Evite em projetos que vão para produção

Aprofundando: a JVM e o ciclo de build

Mensagem de erro	Causa	Solução
`'java' não é reconhecido`	JDK fora do `PATH`	Reinstale marcando *Add to PATH* ou ajuste o `PATH`
`JAVA_HOME is not defined correctly`	Variável aponta para pasta errada	Aponte para a pasta raiz do JDK (sem `\bin`)
`invalid target release: 25`	Maven rodando com JDK antigo	Confira `java -version` no mesmo terminal
`Permission denied: ./mvnw`	Arquivo sem permissão de execução	`chmod +x mvnw`

10

Você já conhece o npm: agora o Maven

Você já conhece o npm: agora o Maven

Na pasta Front-end você usou Node.js, npm e o `package.json`. No Java, cada uma dessas peças tem um equivalente com o mesmo papel. Se você entendeu o npm, já entendeu metade do Maven.

JavaScript / React (pasta Front-end)	Java / Spring (esta pasta)
Node.js	JDK (e a JVM dentro dele)
npm	Maven (`mvnw`)
`package.json`	`pom.xml`
registro npm (npmjs.com)	Maven Central
`node_modules/` / `dist/`	`~/.m2` / `target/`
`npm install`	`./mvnw dependency:resolve` (o build já faz sozinho)
`npm run dev`	`./mvnw spring-boot:run`
`npm run build`	`./mvnw package`

npm x Maven: diferenças que confundem

npm

- Dependências ficam dentro do projeto (`node_modules`)
- Scripts livres no `package.json`
- Versões com `^` e `~`

vs

Maven

- Dependências ficam em uma pasta global (`~/.m2`)
- Fases fixas: compile, test, package...
- Versões exatas, geridas pelo parent do Spring Boot

11

Quiz de revisão

Quiz · questão 1 de 5

Pergunta

Qual peça é suficiente para executar um `.jar` pronto, sem compilar nada?

A JDK

B JRE

C Maven

D javac

Quiz · questão 1 de 5

Resposta

Qual peça é suficiente para executar um `.jar` pronto, sem compilar nada?

A JDK

✓ JRE

C Maven

D javac

Por quê: A JRE traz a JVM e as bibliotecas padrão. O JDK só é necessário para compilar.

Pergunta

Por que a disciplina usa o JDK 25?

A. É a única versão compatível com o VS Code

B. É a versão LTS mais recente

C. O Spring Boot 4 só roda no 25

D. É a versão que vem no Windows

Quiz · questão 2 de 5

Resposta

Por que a disciplina usa o JDK 25?

A. É a única versão compatível com o VS Code

✓ É a versão LTS mais recente

C. O Spring Boot 4 só roda no 25

D. É a versão que vem no Windows

Por quê: O 25 é LTS (suporte longo). O Spring Boot 4 exige no mínimo Java 17.

Pergunta

O que o Maven Wrapper (`mvnw`) resolve?

A. Dispensa instalar o Maven e fixa a versão usada pela turma

B. Substitui o JDK

C. Instala o VS Code

D. Gera o Dockerfile

Resposta

O que o Maven Wrapper (`mvnw`) resolve?

✓ Dispensa instalar o Maven e fixa a versão usada pela turma

B Substitui o JDK

C Instala o VS Code

D Gera o Dockerfile

Por quê: Ele baixa a versão correta do Maven na primeira execução.

Pergunta

O erro `permission denied: ./mvnw` no Linux/macOS se resolve com:

A `sudo java -jar`

B `chmod +x mvnw`

C `npm install`

D Reinstalar o JDK

Resposta

O erro `permission denied: ./mvnw` no Linux/macOS se resolve com:

A `sudo java -jar`

✓ `chmod +x mvnw`

C `npm install`

D Reinstalar o JDK

Por quê: O arquivo perdeu a permissão de execução, comum ao baixar o projeto como .zip.

Pergunta

Para que serve a variável `JAVA_HOME`?

A. Guardar o código-fonte

B. Indicar a pasta do JDK para Maven, IDE e Spring

C. Definir a porta da API

D. Configurar o GitHub

Resposta

Para que serve a variável `JAVA_HOME`?

A. Guardar o código-fonte

✓ Indicar a pasta do JDK para Maven, IDE e Spring

C. Definir a porta da API

D. Configurar o GitHub

Por quê: Ferramentas leem o JAVA_HOME para saber qual Java usar.

Fechamento

Resumo da aula

- ✓ O **JDK** compila e contém a **JRE**, que contém a **JVM**, que executa o bytecode.
- ✓ O **JDK 25** é a versão LTS atual; configure o `JAVA_HOME` apontando para ele.
- ✓ O **Maven** baixa dependências, compila e empacota; o **Maven Wrapper** (`mvnw`) dispensa instalar o Maven.
- ✓ No VS Code, o **Extension Pack for Java** e o **Spring Boot Extension Pack** cobrem tudo o que vamos usar.

Exercícios

1. Instale o JDK 25 e envie um print do resultado de `java -version`.

2. Explique com suas palavras por que um servidor de produção pode ter só a JRE.

3. Altere o `Ola.java` para receber dois argumentos (nome e curso) e imprimir os dois.

4. Descubra onde o JDK foi instalado no seu sistema e confira se o `JAVA_HOME` aponta para lá.

Para ir além

- adoptium.net: downloads do Eclipse Temurin
- maven.apache.org: documentação do Maven
- code.visualstudio.com/docs/java: Java no VS Code

Repositório da disciplina

Todos os projetos citados estão na pasta PROJETOS

Abrir no GitHub

Links e versões citados podem ter mudado. Confira sempre a documentação oficial.



Obrigado!

Dúvidas sobre a aula 01?

Prof. Dr. João Paulo Aramuni
Engenharia de Software · PUC Minas
Desenvolvimento e Integração de Aplicações Web

GitHub da disciplina
https://github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web